WORLD 1 / JAVASCRIPT

The JavaScript Workshop

Build reliable programs from first values to asynchronous work queues

The workshop turns a list of language features into a sequence of working programs. Start with values and small functions, then trace what happens when your code shares data, calls another operation, or waits for a result. Every lesson pairs a runnable example with a check for understanding and a challenge you can implement.

Choose the Foundations path if you are new to JavaScript. You will write functions, process collections, validate JSON and represent failures. Choose the Application path once you can comfortably read a loop and write a function. It builds from closures and shared data to promises, callback boundaries and a bounded queue.

Run the examples with a modern JavaScript engine such as Node.js 24 or a browser console. Start each example in a fresh environment to avoid redeclaring top-level names. Predict the result, run the example and explain why it behaves as it does before changing an input.

Start with the beginner route if these ideas are new to you. In the game, choose a hero to save your checkpoint answers, code drafts, and quest log. The attached field kit contains the worked examples, starter files, and a quest journal.

Explore the world map · Begin Level 01

World map

Foundations

Start here if you are new to JavaScript. Build small programs that turn raw input into useful records and report failures clearly.

Application

Use familiar language basics to solve the problems that arise when programs share state, pass methods as callbacks and coordinate asynchronous work.

Full Workshop

Work through both zones in order, starting with values and ending by implementing and testing an asynchronous job queue.

The Workbench

01 / Bindings and Values
02 / Comparisons and Defaults
03 / Arrays, Records and Missing Fields
04 / Functions with Clear Contracts
05 / Branches and Loops
06 / Validate Data at the Boundary
07 / Errors and Small Tests

The Dispatch Floor

08 / Closures and Private State
09 / References, Copies and Identity
10 / Transforms and Deterministic Ordering
11 / Promises and Result Order
12 / Async Errors and Stale Results
13 / Method Receivers and Callback Boundaries
14 / Capstone: A Bounded Work Queue

LEVEL 01 / FIELD MANUAL

Bindings and Values

Use const and let deliberately and distinguish changing a binding from mutating a value.

A binding gives a name to a value. Use const when that name should keep referring to the same value and let when the algorithm needs to assign a different one. A count that advances is a good let binding. A settings record that is created once is a good const binding, even if its properties change.

JavaScript strings, numbers, booleans, bigints, symbols, null and undefined are primitive values. A string is immutable, so trimming it produces a new string. Objects and arrays hold mutable properties. Two bindings that refer to the same object observe changes through either binding. Assigning one binding to another does not clone the object.

Declare a binding before using it and choose a name that describes its role. Keep raw input distinct from the validated value. The example uses trim and uppercasing to build a display name while leaving the raw string available for inspection.

Worked example / Level 01

Listing 1.1 JavaScript example

```
const raw = '  dana  ';
const displayName = raw.trim().toUpperCase();
let visits = 0;
visits += 1;
const badge = { name: displayName, visits };
badge.visits += 1;
console.log(raw.length, badge.name, badge.visits); // 8 DANA 2
```

Remember. const prevents reassignment of the binding. Object properties can still change.

Try the encounter

LEVEL 01 / FORGE

The encounter

Checkpoint

Given const badge = { visits: 1 }, what happens when you run badge.visits += 1?

A. The program throws because badge uses const.

B. The property changes to 2 and badge still refers to the same object.

C. A second object is created and assigned to badge.

D. The property stays at 1 because const freezes nested data.

Code challenge

Implement makeBadge(rawName). The input is a string. Remove surrounding whitespace, uppercase the result, and return a fresh record with fields name and visits, with visits starting at 0. makeBadge(' Jo ') should return { name: 'JO', visits: 0 }. Two calls should return separate records.

Level 1 starter

```
function makeBadge(rawName) {
  // Normalize the name and build the record.
  return { name: rawName, visits: 0 };
}
```

Read the debrief · Back to the field manual

LEVEL 01 / EXPLAIN

Debrief and side quest

Checkpoint. B. The assignment targets a property on the object. The badge binding is never reassigned.

Hint one

String methods return a string. Keep the result of each step.

Hint two

Chain rawName.trim().toUpperCase() and use an object literal in the return expression.

Compare your solution

Use a const binding for the cleaned name and create { name, visits: 0 } inside the function. A new object literal on each call prevents two callers from accidentally sharing visit counts.

Side quest

Call makeBadge twice with the same name. Increment the visits property on the first result and check that the second stays at zero.

Continue to Level 02 · World map

LEVEL 02 / FIELD MANUAL

Comparisons and Defaults

Predict strict equality, boolean conditions and nullish defaults.

A comparison is a policy choice. For a product ID, the string '7' and the number 7 might mean different things. Strict equality compares without converting types, so those values are unequal. Convert explicitly at the input boundary when your program expects a number. After conversion, use a predicate such as Number.isFinite or Number.isInteger to check the result.

A boolean condition asks whether a value is truthy. Zero and the empty string are falsy, yet both can be meaningful user choices. The expression left || right substitutes right when left is falsy. The expression left ?? right substitutes right only when left is null or undefined. Pick the condition that matches your input contract.

The Number constructor has conversion rules that may surprise you. Number('') and Number(' ') are zero. When blank means missing, trim and check for an empty string before converting. Explicitly separate a missing value, an invalid value and a valid zero.

Worked example / Level 02

Listing 2.1 JavaScript example

```
const savedVolume = 0;
const retryLimit = undefined;
console.log(savedVolume || 50); // 50
console.log(savedVolume ?? 50); // 0
console.log(retryLimit ?? 3); // 3
console.log('7' === 7, Number('7') === 7); // false true
console.log(Number.isFinite(Number('oops'))); // false
```

Remember. Use strict equality for comparisons and choose a default based on what counts as missing.

Try the encounter

LEVEL 02 / FORGE

The encounter

Checkpoint

A user sets volume to 0. Which expression preserves that setting and defaults to 50 for null or undefined?

A. volume || 50

B. Boolean(volume) ? volume : 50

C. volume ?? 50

D. volume === 0 ? 50 : volume

Code challenge

Implement readLimit(raw, fallback). raw is a string, null or undefined. Return fallback for a nullish or blank input. For any other string, use Number to convert it and return the value if it is a nonnegative integer. Return null for an invalid value. Check '0', ' 12 ', ' ', '3.5' and 'oops'.

Level 2 starter

```
function readLimit(raw, fallback) {
  // Handle missing input before converting or checking the number.
  return fallback;
}
```

Read the debrief · Back to the field manual

LEVEL 02 / EXPLAIN

Debrief and side quest

Checkpoint. C. Zero is falsy but is neither null nor undefined. The nullish operator preserves it.

Hint one

Test raw === null || raw === undefined before calling trim.

Hint two

An empty trimmed string needs the fallback. Then convert with Number and test Number.isInteger(value) && value >= 0.

Compare your solution

Treat missing and blank inputs together. Convert nonblank input once, validate it, and return either that number or null. The caller can distinguish an invalid string from a missing preference.

Side quest

Try the string '0x10'. If only decimal digits are allowed, add a digit-only check before Number and describe the changed contract.

Continue to Level 03 · World map

LEVEL 03 / FIELD MANUAL

Arrays, Records and Missing Fields

Represent ordered collections with arrays and named attributes with objects.

An array is an ordered collection. Index zero is the first position. A dense array of length three has elements at indexes zero through two. Sparse arrays can have missing elements within their length. A plain object models named fields such as a shipment's id and quantity. Keep records inside a dense array when both field names and collection order matter.

Read a known property with dot notation. Use brackets for a property name computed at runtime. A missing property normally reads as undefined, which is also a value a record can store deliberately. Object.hasOwn(record, key) answers whether the key is present directly on the object. It avoids mistaking an inherited property for user data.

Choose a clear lookup result. A search for one matching record can return undefined or null when no item matches. Document which one your function returns. The example returns null to make a missing shipment explicit before a caller reads its fields.

Worked example / Level 03

Listing 3.1 JavaScript example

```
const shipments = [
  { id: 'A1', quantity: 3 },
  { id: 'B4', quantity: 0 },
];
function findShipment(id) {
  return shipments.find(shipment => shipment.id === id) ?? null;
}
const shipment = findShipment('B4');
const key = 'quantity';
console.log(Object.hasOwn(shipment, key), shipment[key]); // true 0
console.log(findShipment('X0')); // null
```

Remember. Use arrays for ordered items, records for named facts, and an explicit result for a missing lookup.

Try the encounter

LEVEL 03 / FORGE

The encounter

Checkpoint

A record contains { quantity: undefined }. What does Object.hasOwn(record, 'quantity') report?

A. false, because reading quantity gives undefined.

B. undefined, because hasOwn returns the value of the property.

C. true, because the property exists on the record.

D. An error, because object fields cannot contain undefined.

Code challenge

Implement quantitiesFor(shipments, id). Return a new array containing the quantity of each record with a matching id, in the original order. Keep zero quantities. Return an empty array when there are no matches and leave the input untouched.

Level 3 starter

```
function quantitiesFor(shipments, id) {
  const quantities = [];
  // Inspect each shipment and append matching quantities.
  return quantities;
}
```

Read the debrief · Back to the field manual

LEVEL 03 / EXPLAIN

Debrief and side quest

Checkpoint. C. Object.hasOwn checks property presence. The stored value is a separate fact.

Hint one

Compare the shipment's id with the requested id. Compare IDs, not quantities.

Hint two

A for…of loop over shipments can push shipment.quantity into the result after a strict equality check.

Compare your solution

Collect into a fresh array and append a quantity whenever shipment.id === id. A zero quantity is still a result. The original array and its records are unchanged.

Side quest

Add a separate lookup that returns the first matching record or null. Explain why it should stop at the first match.

Continue to Level 04 · World map

LEVEL 04 / FIELD MANUAL

Functions with Clear Contracts

Write functions with explicit inputs, outputs and valid ranges.

A function contract tells the caller which arguments it accepts, what it returns and whether it changes anything the caller can observe. Start with the result. A function that computes an invoice total can return a number, while code outside that function decides whether to display, save or transmit it. Keeping those jobs separate makes the arithmetic easy to test.

Parameters receive values from the caller. A default parameter applies when that argument is undefined or omitted. An explicit null is a value and does not trigger the default. For an optional record of settings, consider whether every supplied field has a meaningful null value before choosing defaults.

A function ends at return. If it falls off the end, the result is undefined. Use explicit returns for alternatives, particularly when one condition yields no usable result. Test the edges of any promised range. For money-like arithmetic in this lesson, use whole cents and assume the result fits in a safe integer.

Worked example / Level 04

Listing 4.1 JavaScript example

```
function lineTotal(unitCents, count = 1) {
  return unitCents * count;
}
function invoiceLabel(unitCents, count) {
  const cents = lineTotal(unitCents, count);
  return cents + ' cents';
}
console.log(invoiceLabel(275, 3)); // 825 cents
console.log(lineTotal(275), lineTotal(275, 0)); // 275 0
```

Remember. Make the function's inputs and return value easy to predict, then put display or I/O at the boundary.

Try the encounter

LEVEL 04 / FORGE

The encounter

Checkpoint

Given function charge(cents, count = 1), what count does charge(200, 0) receive?

A. 0

B. 1

C. undefined

D. null

Code challenge

Implement shippingCents(subtotalCents, delivery = 'standard'). Inputs are a nonnegative integer and either 'standard' or 'express'. Express always costs 1200 cents. Standard costs 500 cents when the subtotal is below 5000 and zero at or above 5000. Return the number of cents.

Level 4 starter

```
function shippingCents(subtotalCents, delivery = 'standard') {
  // Handle express and the free standard shipping boundary.
  return 500;
}
```

Read the debrief · Back to the field manual

LEVEL 04 / EXPLAIN

Debrief and side quest

Checkpoint. A. An explicit zero is supplied. The default applies only to an omitted or undefined argument.

Hint one

Check the delivery mode before evaluating the standard shipping threshold.

Hint two

Return 1200 immediately for express, otherwise use subtotalCents >= 5000 to choose between 0 and 500.

Compare your solution

Test standard at 4999 and 5000, express at 5000, and a call with the second argument omitted. These checks distinguish the delivery rule from the default and its boundary.

Side quest

Write a second function that formats the result for display while leaving shippingCents focused on calculation.

Continue to Level 05 · World map

LEVEL 05 / FIELD MANUAL

Branches and Loops

Trace decisions and iterate without silently dropping valid data.

Branches give names to conditions your program treats differently. A guard clause can reject a special case before the main work begins. In a loop, continue skips the remaining statements for the current item, while break ends the entire loop. Confusing the two can leave later records unprocessed.

Use for…of to walk array values in order. A for…in loop instead enumerates property keys and can see inherited enumerable properties. If an algorithm needs array indexes, a conventional for loop or array.entries() makes that need explicit.

Track exactly what your loop accumulates. A running total has an initial state and a rule for each item. Try a list where an excluded item comes before a valid one. It often exposes a misplaced break or early return that all-valid examples miss.

Worked example / Level 05

Listing 5.1 JavaScript example

```
const tasks = [
  { done: true, minutes: 10 },
  { done: false, minutes: 20 },
  { done: false, minutes: 0 },
];
let pendingMinutes = 0;
for (const task of tasks) {
  if (task.done) continue;
  pendingMinutes += task.minutes;
}
console.log(pendingMinutes); // 20
```

Remember. Choose between skip and stop deliberately, and trace each iteration that changes the result.

Try the encounter

LEVEL 05 / FORGE

The encounter

Checkpoint

What changes if continue is replaced with break in the example?

A. The same result is printed because both statements skip done tasks.

B. Only the last task is processed.

C. The loop ends at the first done task, leaving the total at 0.

D. The task is removed from the input array.

Code challenge

Implement pendingSummary(tasks). Each task has a boolean done and a nonnegative integer minutes. Return { count, minutes } for unfinished tasks only. Count unfinished tasks that take zero minutes. Return { count: 0, minutes: 0 } for an empty array.

Level 5 starter

```
function pendingSummary(tasks) {
  let count = 0;
  let minutes = 0;
  // Update both totals for unfinished tasks.
  return { count, minutes };
}
```

Read the debrief · Back to the field manual

LEVEL 05 / EXPLAIN

Debrief and side quest

Checkpoint. C. The first task is done. break exits the loop immediately, so the following pending task never adds its minutes.

Hint one

Check done independently from minutes. Zero minutes does not mean complete.

Hint two

Skip completed tasks. For each remaining task, increment count and add task.minutes.

Compare your solution

Initialize both totals to zero, visit each task and skip only tasks marked done. A zero-minute unfinished task increments count without changing minutes.

Side quest

Return a second count for unfinished tasks over 30 minutes and test tasks at exactly 30 and 31.

Continue to Level 06 · World map

LEVEL 06 / FIELD MANUAL

Validate Data at the Boundary

Separate valid JSON syntax from the shape a program expects.

JSON is a text format. JSON.parse turns valid text into JavaScript data and throws on malformed syntax. The resulting value can be an object, an array, a number, a string, a boolean or null. A successful parse says nothing about whether the value fits your application.

Validation should name the expected shape. For a profile record, reject null before reading properties and reject arrays. Since typeof null returns 'object', check for null explicitly. Use Object.hasOwn to require fields supplied by the record itself, then check each field's type and value.

Normalize only after validating the values you intend to transform. A numeric name cannot be trimmed, and converting it to a string might silently accept a bad payload. Returning a fresh record with only the accepted fields gives later code a smaller, predictable surface to work with.

Worked example / Level 06

Listing 6.1 JavaScript example

```
function parseProfile(text) {
  const value = JSON.parse(text);
  if (value === null || typeof value !== 'object' || Array.isArray(value)) {
    return null;
  }
  if (!Object.hasOwn(value, 'name') || typeof value.name !== 'string') {
    return null;
  }
  const name = value.name.trim();
  return name === '' ? null : { name };
}
console.log(parseProfile('{"name":"  Lina  ","extra":true}')); // { name: 'Lina' }
console.log(parseProfile('[]')); // null
```

Remember. Parse the text, validate the data, then produce the value your program expects.

Try the encounter

LEVEL 06 / FORGE

The encounter

Checkpoint

JSON.parse('null') succeeds. What should a validator for a profile record do next?

A. Accept null because the JSON parser succeeded.

B. Read value.name because typeof null is 'object'.

C. Call JSON.parse again to turn null into a record.

D. Reject null before reading fields.

Code challenge

Implement decodeItem(text) for valid JSON text. Accept a non-array record with its own string id and nonnegative integer count. The id must be nonempty after trimming. Return a fresh { id, count } with the trimmed id. For valid JSON of any other shape, return null. Malformed JSON may throw in this lesson.

Level 6 starter

```
function decodeItem(text) {
  const value = JSON.parse(text);
  // Validate record and fields, then normalize the ID.
  return null;
}
```

Read the debrief · Back to the field manual

LEVEL 06 / EXPLAIN

Debrief and side quest

Checkpoint. D. JSON null is valid syntax, but a null value does not satisfy a record contract and cannot be used for property access.

Hint one

Check the top-level type and shape, then require id and count with Object.hasOwn.

Hint two

Trim only a string id, and test Number.isInteger(value.count) && value.count >= 0. Build a fresh record.

Compare your solution

Test a valid count of zero and invalid inputs including null, an array, an empty ID and a numeric string count. Those distinguish shape checking from truthiness and automatic conversion.

Side quest

Wrap only the JSON.parse call in a try/catch and return different error labels for malformed JSON and an invalid item shape.

Continue to Level 07 · World map

LEVEL 07 / FIELD MANUAL

Errors and Small Tests

Distinguish expected validation failures from exceptions and test both paths.

An error boundary defines what happens when an operation cannot produce its usual result. A user can predictably enter invalid data, so an application can return a structured failure that explains how to recover. An unexpected programming problem may deserve an exception with its original error and stack.

A try/catch catches exceptions raised while the protected code is running. If a catch block turns every exception into 'invalid input', a defect in unrelated code becomes hard to diagnose. Put only the operation you intend to translate inside that boundary and check the error type where the distinction matters.

Write small checks around observable behavior. A success example checks the returned value. A failure example checks how the caller can identify the failure. Use exact inputs to cover a boundary, such as '0' and '-1'. Console output helps inspection, but an assertion that throws on mismatch makes a broken example visible to automated tools.

Worked example / Level 07

Listing 7.1 JavaScript example

```
function readJson(text) {
  try {
    return { ok: true, value: JSON.parse(text) };
  } catch (error) {
    if (!(error instanceof SyntaxError)) throw error;
    return { ok: false, reason: 'syntax' };
  }
}
const good = readJson('0');
const bad = readJson('{');
if (!good.ok || good.value !== 0) throw new Error('Expected zero');
if (bad.ok || bad.reason !== 'syntax') throw new Error('Expected syntax error');
console.log('Both checks passed');
```

Remember. Keep error translation narrow and make your tests assert a precise result.

Try the encounter

LEVEL 07 / FORGE

The encounter

Checkpoint

Why does readJson inspect the exception type and rethrow an unexpected error?

A. So unrelated failures are not reported as invalid JSON.

B. Because try/catch can catch only SyntaxError.

C. Because all thrown values are automatically converted to null.

D. So parsing can keep going after malformed JSON.

Code challenge

Implement parseCount(text) for string input. Return { ok: false, reason: 'syntax' } for malformed JSON. For valid JSON that is not a nonnegative integer number, return { ok: false, reason: 'count' }. Return { ok: true, value } otherwise. Test '0', '-1', '1.5', '"2"' and '{'.

Level 7 starter

```
function parseCount(text) {
  let value;
  // Parse with a narrow catch. Validate outside that catch.
  return { ok: true, value };
}
```

Read the debrief · Back to the field manual

LEVEL 07 / EXPLAIN

Debrief and side quest

Checkpoint. A. The boundary translates an expected parse failure. An unexpected error keeps its original diagnostic.

Hint one

Put the JSON.parse assignment inside the try block. Keep the integer check afterward.

Hint two

Translate SyntaxError to the syntax result and rethrow other errors. A nonnegative integer satisfies Number.isInteger(value) && value >= 0.

Compare your solution

Parsing and validation describe different failures. Returning a distinct reason for each lets a caller handle both while preserving unexpected exceptions.

Side quest

Write a tiny assertion helper, run the five sample checks and show the failing case's input if any check fails.

Continue to Level 08 · World map

LEVEL 08 / FIELD MANUAL

Closures and Private State

Use lexical scope to expose a small interface and test whether instances share state.

A function can read bindings from the scope in which it was created. When the function travels elsewhere, it keeps access to those bindings. This is a closure. Where the caller invokes the function does not determine which local variables it sees.

A closure keeps access to a binding, so it can observe a later assignment to that binding. Separate calls to a factory create separate sets of local bindings. That lets a factory return operations over private state without publishing that state directly.

A callback created inside a loop can keep the loop's current values. A let binding in a for-loop initializer creates separate per-iteration bindings for that loop. For a factory like the one below, use a small test with two instances. A single instance may pass even if state has accidentally been moved into a shared outer scope.

Worked example / Level 08

Listing 8.1 JavaScript example

```
function makeCounter(start = 0) {
  let count = start;
  return {
    increment: () => ++count,
    read: () => count,
  };
}
const first = makeCounter();
const second = makeCounter(10);
console.log(first.increment(), first.increment()); // 1 2
console.log(second.read()); // 10
```

Remember. Closures preserve access to the bindings at their creation site, including private mutable state.

Try the encounter

LEVEL 08 / FORGE

The encounter

Checkpoint

Why does incrementing first leave second unchanged?

A. Object methods automatically copy all their inputs.

B. Each makeCounter call creates its own count binding.

C. Arrow functions can read only constants.

D. JavaScript reinitializes count after each method call.

Code challenge

Implement makeTicketDispenser(prefix). Return an object with next() and issued(). next returns strings such as 'LAB-1', 'LAB-2' and increases the count. issued returns how many tickets the instance has given out. New instances must start independently.

Level 8 starter

```
function makeTicketDispenser(prefix) {
  let count = 0;
  return {
    next() { /* Update count and format the ticket. */ },
    issued() { return count; },
  };
}
```

Read the debrief · Back to the field manual

LEVEL 08 / EXPLAIN

Debrief and side quest

Checkpoint. B. Each call has a distinct local count. The returned functions close over the count from their own factory call.

Hint one

Place count inside the factory so no other factory call shares it.

Hint two

Increment before formatting the prefix, a hyphen and the new count.

Compare your solution

Call next twice on one dispenser and once on another with a different prefix. Check each issued result and ensure calling issued never changes the count.

Side quest

Return the next function by itself and invoke it as a callback. Explain how it still reaches its own count.

Continue to Level 09 · World map

LEVEL 09 / FIELD MANUAL

References, Copies and Identity

Update nested data without surprising callers who hold older references.

Comparing objects with strict equality compares identity. Two objects with the same properties are distinct if they were created separately. An assignment, a parameter and a property can all hold the same object value. A change through any of those references can be visible through the others.

Object spread copies own enumerable properties into a new object. Array spread collects values from an iterable into a new array. Both are shallow. A nested object remains shared with the source. To change a nested field while preserving the previous version, copy the containers along the path to the field.

A Map keeps key-to-value associations without requiring every key to be a string. A Set keeps unique members. Object keys used in either are compared by identity. If two records represent the same product by ID, use the ID as the key. A Set of record objects would keep both separate objects.

Worked example / Level 09

Listing 9.1 JavaScript example

```
const before = { id: 'P7', settings: { theme: 'light' }, tags: ['saved'] };
const after = {
  ...before,
  settings: { ...before.settings, theme: 'dark' },
};
console.log(before.settings.theme, after.settings.theme); // light dark
console.log(before.tags === after.tags); // true
const byId = new Map([[after.id, after]]);
console.log(byId.get('P7') === after); // true
```

Remember. Copy each container you need to change. A shallow copy preserves references to nested values.

Try the encounter

LEVEL 09 / FORGE

The encounter

Checkpoint

If you copy only the outer record and then assign copy.settings.theme, what happens to before.settings.theme?

A. It changes too because both settings properties refer to the same object.

B. It is reset to undefined.

C. It stays unchanged because object spread recursively copies nested data.

D. The assignment throws because settings is a nested property.

Code challenge

Implement setTheme(profile, theme) for plain data records with own enumerable fields. Return new profile and settings records with settings.theme changed. Preserve the other fields, leave the original unchanged and keep the tags array shared.

Level 9 starter

```
function setTheme(profile, theme) {
  // Copy the records along the path to theme.
  return { ...profile };
}
```

Read the debrief · Back to the field manual

LEVEL 09 / EXPLAIN

Debrief and side quest

Checkpoint. A. The outer record is new, but the settings object was copied as a reference. Copy settings too before changing its theme.

Hint one

Check that both the outer profile and its settings have new identities.

Hint two

Use a nested spread and put theme after the existing settings properties so it replaces the prior theme.

Compare your solution

Compare the original and result at each layer. The result and its settings should differ in identity, while result.tags === profile.tags should be true.

Side quest

Write addTag(profile, tag) that creates a new tags array only when tag is absent. Decide what identity the function should return when nothing changes.

Continue to Level 10 · World map

LEVEL 10 / FIELD MANUAL

Transforms and Deterministic Ordering

Build a data pipeline with explicit selection, transformation and sorting rules.

Use filter to choose elements and map to transform each selected element. Use reduce or a loop when several elements contribute to one result. Split a complicated pipeline at points where you can name the intermediate value and test it. A named projection makes it clearer whether you intend to sort original records or display summaries.

Array.prototype.sort mutates the array it receives. Copy an input array first if callers still rely on its original order. The default sort orders string representations. A numeric ascending comparator uses a - b and must return a negative number, zero or a positive number according to the pair's order.

A stable sort preserves the original relative order of items that compare equal. When the requirement calls for a second ordering rule, encode that rule in the comparator. Avoid using a boolean as a comparator result. It cannot express all three outcomes consistently.

Worked example / Level 10

Listing 10.1 JavaScript example

```
const jobs = [
  { id: 'D', priority: 2, active: true },
  { id: 'A', priority: 10, active: true },
  { id: 'F', priority: 1, active: false },
];
const active = jobs.filter(job => job.active);
active.sort((a, b) => b.priority - a.priority);
const ids = active.map(job => job.id);
console.log(ids.join(',')); // A,D
console.log(jobs[0].id); // D
```

Remember. Sorting mutates its input. Sort a new array with a comparator that states the ordering rule.

Try the encounter

LEVEL 10 / FORGE

The encounter

Checkpoint

Why does sorting active leave jobs in its original order?

A. The comparator disables mutation.

B. Only an array with more than two elements can be mutated.

C. filter created a separate outer array.

D. Sorting an array of records never changes their positions.

Code challenge

Implement leaderboard(players). Select records with active === true, sort by numeric score descending and return a new array of their names. For equal scores, preserve their input order. Leave the input order and records untouched.

Level 10 starter

```
function leaderboard(players) {
  // Select into a new array, sort, and project to names.
  return [];
}
```

Read the debrief · Back to the field manual

LEVEL 10 / EXPLAIN

Debrief and side quest

Checkpoint. C. filter returns a new outer array. Its elements are shared records, but rearranging that array does not rearrange jobs.

Hint one

Run filter first. It gives you a fresh outer array to sort.

Hint two

Use (a, b) => b.score - a.score, then map each remaining record to name.

Compare your solution

Test a score of 10 beside 2, two equal scores, and an inactive leader. Also verify that the order of the input records did not change.

Side quest

Aggregate player scores into a Map keyed by team ID and describe how you would break ties explicitly.

Continue to Level 11 · World map

LEVEL 11 / FIELD MANUAL

Promises and Result Order

Follow settlement order and preserve input order when coordinating independent work.

A promise represents a future fulfillment or rejection. A callback registered with then runs after the current synchronous code completes, even if the promise is already fulfilled. A thrown error in that callback rejects the next promise in the chain. Return a promise from the callback when the next step must wait for it.

Promise.all collects results in input order. A task that completes first does not become the first result unless it was the first input. The aggregate rejects if any input rejects. That does not cancel the other operations, which may still finish and produce their own effects.

Choose deliberately whether work should start together or one step at a time. Independent reads can overlap. A second operation that requires a first result must wait for it. Constructing an array of promises usually starts the operations before Promise.all is called. Passing an array of functions alone does not start their work.

Worked example / Level 11

Listing 11.1 JavaScript example

```
function later(value, delay) {
  return new Promise(resolve => setTimeout(() => resolve(value), delay));
}
const first = later('first input', 20);
const second = later('second input', 0);
Promise.all([first, second])
  .then(values => console.log(values.join(' | ')))
  .catch(error => console.error(error));
console.log('Both operations started');
// Both operations started
// first input | second input
```

Remember. Promise.all orders results by input position and reports a rejection without cancelling other operations.

Try the encounter

LEVEL 11 / FORGE

The encounter

Checkpoint

Suppose the second input settles before the first. Where does the second value appear in the fulfilled Promise.all result?

A. Index 0, because it settled first.

B. Index 1, because it was the second input.

C. The order depends on the host's timer precision.

D. It appears only after calling Promise.allSettled instead.

Code challenge

Implement loadInOrder(ids, load). For each ID, call the supplied async load(id) once and return a promise for the values in input order. Start the calls without awaiting one before starting the next. If a load rejects, let the returned promise reject.

Level 11 starter

```
async function loadInOrder(ids, load) {
  // Start all independent operations and collect their results.
  return [];
}
```

Read the debrief · Back to the field manual

LEVEL 11 / EXPLAIN

Debrief and side quest

Checkpoint. B. Promise.all retains each input's position. The result becomes available once all inputs fulfill.

Hint one

An array of IDs can map to an array of promises.

Hint two

Use Promise.all on ids.map(id => load(id)). Promise.all preserves the input positions.

Compare your solution

Check that load is invoked for every ID before any of the pending promises resolves. Complete them in reverse order and verify the returned array still follows ids.

Side quest

Write a separate variant using Promise.allSettled that reports failures beside their input IDs.

Continue to Level 12 · World map

LEVEL 12 / FIELD MANUAL

Async Errors and Stale Results

Prevent older asynchronous results from replacing newer state.

An async function returns a promise. Await pauses that function until the awaited value settles and leaves the host free to run other work. If the awaited promise rejects, await throws inside the async function, where a surrounding try/catch can handle it. Other calls to the same async function may have started by then.

A search box often needs latest-request-wins behavior. If a first search is slow and a second search is fast, updating the display whenever a request completes can put old results back on screen. Give each request a unique token and check it just before publishing either a result or an error.

Suppressing a stale result and cancelling work are separate operations. A token prevents a stale update but does not stop the underlying operation. An AbortSignal can request cancellation from APIs that agree to honor it. Keep the token check if a cancellation request may arrive too late or if the dependency ignores the signal.

Worked example / Level 12

Listing 12.1 JavaScript example

```
function latestOnly(load, publish) {
  let current;
  return async query => {
    const mine = Symbol();
    current = mine;
    let outcome;
    try {
      outcome = { ok: true, value: await load(query) };
    } catch (error) {
      outcome = { ok: false, error };
    }
    if (mine === current) publish(outcome);
  };
}
const load = async query => {
  if (query === 'old') throw new Error('Old request failed');
  return query;
};
const search = latestOnly(load, outcome => console.log(outcome));
Promise.all([search('old'), search('new')]);
// { ok: true, value: 'new' }
```

Remember. Check request identity after await to guard the write. Cancellation requires cooperation from the dependency.

Try the encounter

LEVEL 12 / FORGE

The encounter

Checkpoint

What does the token check guarantee in latestOnly?

A. The underlying older request is cancelled automatically.

B. The newest request necessarily completes first.

C. Only the request that is current when it handles its outcome can publish.

D. The older async function stops existing before its promise settles.

Code challenge

Implement makeLatest(load, publish). It returns an async function run(query). Call load(query) for each run. If the run is still most recent when it handles the outcome, publish { ok: true, value } for fulfillment or { ok: false, error } for rejection or a synchronous throw. Ignore older outcomes. The run promise fulfills after handling either outcome. Assume publish does not throw.

Level 12 starter

```
function makeLatest(load, publish) {
  let current;
  return async function run(query) {
    // Store a new token and form an outcome after the load.
  };
}
```

Read the debrief · Back to the field manual

LEVEL 12 / EXPLAIN

Debrief and side quest

Checkpoint. C. The check guards publication after the await. It makes no request to stop the load itself.

Hint one

Create and store a unique Symbol token synchronously when run starts, before the first await.

Hint two

Call and await load(query) inside try/catch. Form an outcome in either case and publish it only if the token is still current.

Compare your solution

Try a slow success followed by a fast success, then a slow rejection followed by a fast success. The first call should not publish anything in either test.

Side quest

Pass an AbortSignal to a cancellation-aware loader and abort the preceding signal when a new run starts. Keep the token check and test a loader that ignores the signal.

Continue to Level 13 · World map

LEVEL 13 / FIELD MANUAL

Method Receivers and Callback Boundaries

Preserve the intended this when a method crosses a callback boundary.

For an ordinary method call such as ledger.add(3), the property access supplies ledger as this. Evaluating ledger.add alone produces a function value. It does not package the receiver. Passing that function to a callback-taking API can lose the receiver when the API later invokes it.

Use bind when an ordinary function should always use a particular receiver. A wrapper arrow can also call the method with its receiver by writing value => ledger.add(value). Arrow functions themselves inherit this from their creation context. Assigning an arrow as an object property does not make it receive that object as this.

Choose a callback contract that fits the interface. A caller that needs only one operation can receive a function rather than the full object. A test should invoke the returned function as a standalone callback and verify where changes landed. Testing only direct method calls misses receiver loss.

Worked example / Level 13

Listing 13.1 JavaScript example

```
const ledger = {
  total: 0,
  add(amount) {
    this.total += amount;
    return this.total;
  },
};
function apply(operation, amount) {
  return operation(amount);
}
const addToLedger = ledger.add.bind(ledger);
console.log(apply(addToLedger, 7)); // 7
console.log(ledger.total); // 7
```

Remember. An ordinary function's this comes from how it is called. Bind or wrap a method before passing it as a standalone callback.

Try the encounter

LEVEL 13 / FORGE

The encounter

Checkpoint

What does ledger.add.bind(ledger) return?

A. A callback that invokes add with ledger as its receiver.

B. A copy of ledger with a new total field.

C. The return value of calling add immediately.

D. An arrow function whose receiver changes at each call.

Code challenge

Implement makeRecorder(ledger). ledger has an add(amount) method that depends on this and returns its updated total. Return a standalone callback that forwards amount to ledger.add and returns the method's result. Creating the callback must not change the total.

Level 13 starter

```
function makeRecorder(ledger) {
  // Return a callback that keeps the correct receiver.
  return ledger.add;
}
```

Read the debrief · Back to the field manual

LEVEL 13 / EXPLAIN

Debrief and side quest

Checkpoint. A. bind creates a function with the receiver fixed. The method body runs later when the bound function is invoked.

Hint one

Read the difference between returning a function and calling it now.

Hint two

Either bind ledger.add to ledger or return amount => ledger.add(amount).

Compare your solution

Pass the result to a helper that calls operation(amount). The total on the original ledger should change exactly once and the callback should return the new total.

Side quest

Create recorders for two separate ledgers and alternate calls. Explain how the callbacks keep their targets separate.

Continue to Level 14 · World map

LEVEL 14 / FIELD MANUAL

Capstone: A Bounded Work Queue

Run independent asynchronous jobs with a concurrency limit and stable result positions.

Starting every operation at once can exhaust a downstream service or use more memory than expected. A concurrency limit controls how many operations are in progress. A worker claims one job, awaits it, saves the outcome, then claims another job. A pool of workers advances through the same job list.

In one JavaScript agent, synchronous work between await points does not interleave with another job. Claim the next index and increment the cursor before awaiting. Store each result at the claimed index so results follow input order regardless of which operation finishes first.

Define failure behavior before building the queue. Promise.all rejects as soon as an input rejects, but its other workers can still run. In this capstone each operation's error becomes a result record so workers keep draining and the caller sees every outcome. Check the peak active count in a test. The number of results alone cannot prove the limit.

Worked example / Level 14

Listing 14.1 JavaScript example

```
async function collectLimited(items, limit, work) {
  if (!Number.isInteger(limit) || limit < 1) throw new RangeError('limit');
  const results = new Array(items.length);
  let cursor = 0;
  async function worker() {
    while (cursor < items.length) {
      const index = cursor++;
      try {
        results[index] = { ok: true, value: await work(items[index]) };
      } catch (error) {
        results[index] = { ok: false, error };
      }
    }
  }
  const size = Math.min(limit, items.length);
  await Promise.all(Array.from({ length: size }, () => worker()));
  return results;
}
collectLimited([2, 3, 4], 2, async n => n * n)
  .then(results => console.log(results.map(result => result.value))); // [4, 9, 16]
```

Remember. Claim before awaiting, write results by input index, and measure peak concurrent work in your tests.

Try the encounter

LEVEL 14 / FORGE

The encounter

Checkpoint

Where should the worker advance the shared cursor?

A. After awaiting work so other workers see the same current position.

B. Before awaiting work, when it claims the position.

C. Only after the entire pool has completed.

D. In a timer callback shared by all workers.

Code challenge

Implement runQueue(tasks, limit). Each task is a zero-argument function that returns a promise or throws synchronously. Reject with RangeError for a limit that is not a positive integer. Run at most limit tasks at once and return one result per input in input order. Use { ok: true, value } or { ok: false, error }, and continue processing after a failure. The empty queue returns [].

Level 14 starter

```
async function runQueue(tasks, limit) {
  // Validate the limit, then create workers that share a next index.
  return [];
}
```

Read the debrief · Back to the field manual

LEVEL 14 / EXPLAIN

Debrief and side quest

Checkpoint. B. Claim and advance in a synchronous step. After an await, another worker may have observed the old position.

Hint one

Use a shared cursor and give each worker an async loop. Claim an index before calling the task.

Hint two

Catch both sync throws and promise rejections by calling and awaiting the task in one try block. Store the outcome at its original index.

Compare your solution

Use test tasks that signal when they start and resolve only when you release them. Check that no more than limit run at a time, that a failed task does not stop later ones and that reversing completion order does not change result positions.

Side quest

Add a signal that stops claiming new work when aborted and define what outcome unstarted tasks receive. Decide separately how already-running tasks can cooperate with cancellation.

Revisit the world map · Return to Level 01